

Data-Driven Dynamic Pricing in a Competitive E-Commerce Simulation

Comparing Gradient-Boosted and Large Language Model Pricing Agents

MSc Project Specification and Design Proposal (COMP702)

[Your name]

[Your student ID]

Statement of ethical compliance

This project falls into data category B and participant category 0 (B0). It uses publicly available, freely licensed datasets in aggregate. One of these, UCI Online Retail II, consists of anonymised real consumer transaction records from an open source, which places the project in data category B. No human participants are involved in any activity, which gives participant category 0. I confirm that I will follow the School's ethical guidance, that I will confirm the exact data category (A or B) with my supervisor, and that I will update this statement and consult my supervisor before any change of plan that involves human participants.

Project description

Dynamic pricing, the continuous adjustment of product prices in response to market conditions, is one of the most commercially significant applications of data science in e-commerce. A seller must balance profit margin, competitor prices, demand, inventory and seasonality at the same time, in a market that is constantly shifting.

This project builds a custom, Gymnasium-compatible simulation of a competitive e-commerce market, calibrated using real UK retail data, and uses it to study automated pricing. The pricing task is framed as a sequential decision problem in which an agent observes the market state and chooses a price action in order to maximise profit. Within the simulation the project implements and compares pricing agents of increasing sophistication: a ladder of five rule-based baselines, an industry-standard supervised machine learning agent based on gradient boosting, and a Large Language Model (LLM) agent that reasons about the market in natural language. The central question is how well a data-driven numerical optimiser and a language-based reasoner can price, relative to each other and to a theoretical optimum. The project is a hybrid of software implementation (the simulation and tooling) and empirical research (the controlled comparison).

Aims and requirements

Aims

- To design and build a realistic, data-calibrated simulation of a competitive e-commerce market.
- To determine how well a gradient-boosting pricing agent and an LLM pricing agent perform, measured against simple baselines and a theoretical optimum.
- To investigate and critically interpret the pricing behaviours that each approach produces.

Research questions

1. Can a supervised gradient-boosting agent learn a profitable pricing policy in the simulated market, and how close does it come to the theoretical optimum?
2. How does an LLM-based pricing agent, reasoning in natural language, compare with the numerical optimiser across the same market scenarios?
3. What pricing behaviours emerge from each approach, and how can they be interpreted in economically meaningful terms?

Requirements (essential)

- E1. The software shall provide a Gymnasium-compatible market environment with a price-sensitive demand model, inventory, competitor prices and seasonal and day-of-week effects.
- E2. The software shall calibrate the environment's parameters offline from real UK retail datasets.
- E3. The software shall implement five rule-based baseline agents: fixed-price, cost-plus, competitor-matching, random, and an oracle with full knowledge of the true demand.

- E4. The software shall implement a two-stage gradient-boosting predict-then-optimize pricing agent.
- E5. The software shall provide a shared evaluation harness that runs any agent under identical scenarios and seeds and records revenue, gross profit, market share and pricing stability.
- E6. The software shall provide a command-line tool to launch experiments and a results dashboard to compare agents.

Requirements (desirable)

- D1. The software should implement an LLM-based pricing agent that receives the market state as a natural-language prompt and returns a structured pricing decision with reasoning.
- D2. The software should log and allow qualitative analysis of the LLM's reasoning, and support prompt design as an experimental variable.
- D3. The software should support comparing a smaller and a larger LLM to assess whether model capability changes pricing quality.

Key literature and background reading

Dynamic pricing has a substantial research history. Den Boer (2015) surveys the field of pricing and learning, tracing its origins and current directions and motivating the use of adaptive, data-driven methods over rigid rules. A common pattern in production pricing is a two-stage predict-then-optimize pipeline, in which a demand or elasticity model is estimated and a price is then chosen to maximise expected profit; this directly motivates the gradient-boosting agent in this project as a credible industry-standard method rather than a weak baseline.

The sequential nature of pricing is naturally captured by the Markov Decision Process formalism described by Sutton and Barto (2018), and reinforcement-learning approaches to competitive pricing have used exactly this framing (Kastius and Schlosser, 2021; Liu et al., 2019). Although this project does not train a reinforcement-learning agent, that literature informs the design of the environment and the decision to model pricing as an MDP. For the supervised demand model, gradient boosting (Chen and Guestrin, 2016) is well suited to tabular data of this kind. Finally, the use of LLMs as reasoning and decision-making agents is an emerging area (Yao et al., 2023); their application to pricing specifically is comparatively underexplored, which is the originality hook for this project. An early and thorough literature search will be carried out to position the contribution accurately and to avoid overclaiming novelty.

[Expand this section with your own reading. Add several more sources and use them to justify your design choices rather than only to summarise. Ensure every work in the reference list is cited in-text, as it is above.]

Development and implementation summary

Architecture

The design is deliberately modular. Every agent obeys the same simple contract: given the market state, return a price decision. This makes the agents interchangeable and the

comparison between them fair. A single runner drives any agent through identical scenarios and records the outcomes in one place, and the environment is built once and reused for every agent. Real data is used offline, once, only to calibrate the parameters of the environment, and does not train the agents directly. Figure 1 presents the system as five layers, from data and calibration at the top, through the decision agents and the shared interface, to the simulation core and the evaluation and output layer.

System architecture (layered view)

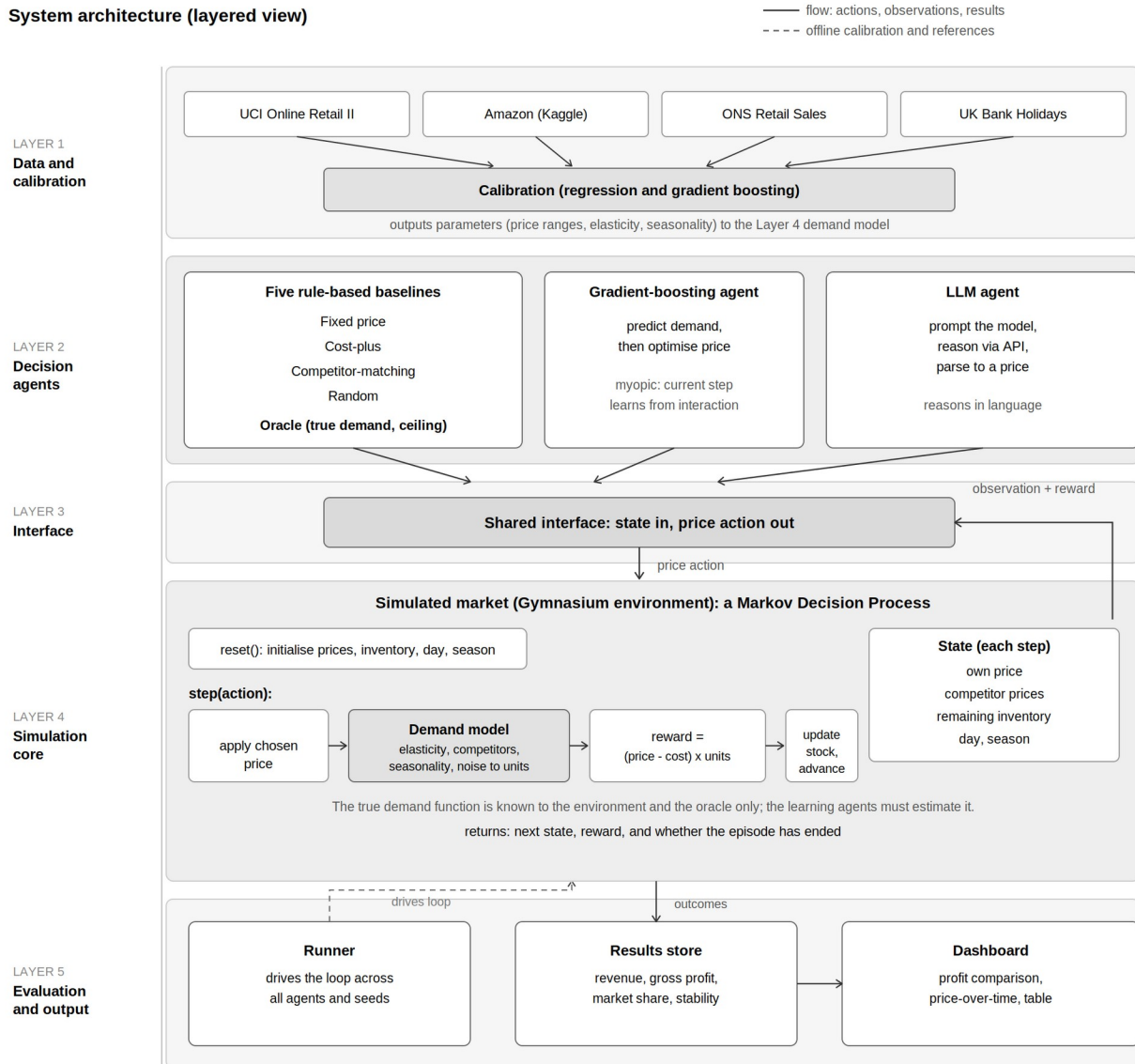


Figure 1: Layered system architecture. Layer 1 calibrates the demand model offline from open datasets, but the true demand function is never shared with the learning agents. Layer 2 holds the agents, which all act through the single interface in Layer 3. Layer 4 is the simulation core, where the step function applies the price, runs the demand model, computes the profit reward, and advances the state. Layer 5 runs the loop across many seeds and turns the recorded outcomes into results and dashboard views.

The simulation environment: Gymnasium and the MDP

The core technical contribution is the simulation environment, built to the Gymnasium standard, the widely used Python interface for sequential decision problems. A simulation provides a safe, fast and fully repeatable laboratory in which thousands of pricing decisions can be played out and every agent tested under identical conditions; Gymnasium is chosen because it is the recognised convention, lets any agent drive the environment the same way, and interoperates with established tooling. Pricing is modelled as a Markov Decision Process, defined as follows:

MDP element	Definition in this project
State	The agent's own current price, competitor prices, the current demand level, remaining inventory, the day of the week, and the season.
Action	A small discrete set of price moves: raise by 5 percent, lower by 5 percent, lower by 10 percent, or hold.
Reward	Gross profit for the period, calculated as (selling price minus unit cost) multiplied by units sold.
Transition	The chosen price drives the demand model (elasticity, competitors, seasonality, noise), which sets units sold; inventory is decremented and time advances.
Discount factor	A value close to 1 (for example 0.99), expressing a preference for long-term over immediate profit.
Episode	One simulated selling horizon, ending at the horizon or at stockout, after which the environment is reset.

Concretely, the environment is a Python class providing two methods. The reset method starts a fresh episode by setting the opening prices, inventory, day and season, and returns the first observation. The step method contains the market logic: it applies the chosen price, runs the demand model to compute units sold given competitor prices, seasonality and stochastic noise, calculates the gross-profit reward, decrements inventory, advances time, decides whether the episode has ended, and returns the new observation, the reward, and that termination signal. A runner loop calls reset and then repeatedly passes each agent's action to step until the episode ends. Because every agent speaks this interface, one loop drives all of them, which is what makes the comparison fair.

Inside step sits the demand model, the economic heart of the simulation: a price-sensitive function grounded in elasticity theory, modulated by competitor prices and seasonal effects and perturbed by noise. Real datasets are used offline to calibrate this function's parameters so that it behaves plausibly, but the true demand function remains a transparent function that the project controls. It defines the environment and is never given to a learning agent. This separation keeps the comparison clean and makes a valid oracle possible, since the oracle alone may use the true demand directly.

Tools and libraries

The project is implemented in Python 3.11 or later, chosen for its data-science ecosystem. The simulation uses Gymnasium; the gradient-boosting agent uses XGBoost or LightGBM with

scikit-learn; data handling uses NumPy and Pandas; the LLM agent uses the OpenAI API, with the specific model chosen for a balance of cost and latency; and results are tracked and visualised with Weights and Biases and Matplotlib.

Workflow

Development proceeds in dependency order, building the environment first as the critical path, then the baselines, then the gradient-boosting agent, then the LLM agent, beginning with a minimal end-to-end pipeline that is progressively enriched. Version control uses Git and GitHub with frequent commits. Experiments are run in sequential batches suited to the development machine (an 8GB Apple M1, running on CPU), with Google Colab available as compute overflow, and every run is logged for reproducibility.

Data sources

The agents do not train on historical data. Real datasets are used only once, offline, to calibrate the simulation so that it behaves realistically. All datasets are public, freely licensed and used in aggregate, obtained from open sources and a public API with no web scraping.

- UCI Online Retail II (Creative Commons BY 4.0): around 1.07 million real UK e-commerce transactions from 2009 to 2011, used to estimate realistic price ranges, sales volumes and demand response. This dataset contains anonymised consumer transaction records; customer identifiers are not used, and only aggregate price and quantity patterns inform calibration. Its presence is the reason the project is classified under data category B.
- Amazon Product Dataset (Kaggle): product listings with prices and categories, used to ground price distributions and cost structures. The specific dataset licence will be confirmed before use.
- ONS UK Retail Sales Index (Open Government Licence): official monthly retail sales by sector, used for seasonal patterns.
- UK Bank Holidays (gov.uk public API, Open Government Licence): used to model holiday demand effects such as the Christmas peak.

All sources will be cited and acknowledged where they form a substantive part of the work.

Testing and evaluation

Technical testing

Unit tests will verify that the environment behaves correctly: that demand falls as price rises and at a plausible magnitude, that inventory depletes correctly, that the reward equals (price minus unit cost) multiplied by units sold, and that reset and step return well-formed observations of the expected shape. Integration testing will confirm that each agent can be driven through the shared interface by the same runner and that results are recorded consistently across agents and seeds. The oracle baseline serves as an additional correctness check: because it has perfect knowledge it should never be beaten, so any agent that beats it indicates a bug in the environment or the evaluation.

Evaluation

All agents are run through the same scenarios, seeds and metrics, namely cumulative revenue, gross profit, market share, and pricing stability (the standard deviation of price changes), averaged across multiple seeds for robustness. Results are interpreted as a ladder: the simplest baselines anchor the floor and the oracle anchors the ceiling, so the share of the theoretical optimum that each method captures gives a meaningful measure of quality rather than an isolated number. The LLM agent additionally enables a qualitative analysis of its logged reasoning. No human participants are used in evaluation; all testing is carried out by the developer.

Project ethics and human participants

No human participants are involved in this project. There is no requirements-analysis activity beyond discussion with my supervisor, and no software evaluation involving other people; all testing and evaluation is carried out by me. The project therefore falls into participant category 0. The data used is public and aggregate, including one dataset of anonymised consumer transactions, which places the project in data category B, giving an overall classification of B0. Should a user evaluation be added later, I will move to the appropriate participant category, use the participant information sheet and consent form, and consult my supervisor first.

Beyond data and participant ethics, automated pricing raises genuine societal considerations, including tacit algorithmic collusion, price discrimination and fairness. These will be acknowledged and discussed as part of the professional and ethical framing of the work, in line with the BCS Code of Conduct.

BCS project criteria

- **Systematic understanding and current problems.** The project engages with current production pricing methods and the emerging area of LLM decision agents, supported by the literature review.
- **Comprehensive understanding of techniques.** It applies gradient boosting, prompt-based LLM reasoning, simulation and MDP design, and a rigorous experimental method with multiple seeds and a fair protocol.
- **Originality in application.** Comparing a numerical optimiser against a language-based reasoner for pricing, on a data-calibrated environment, is the original contribution.
- **Sound judgement under incomplete data.** The validity limits of the simulation, the decision to calibrate rather than define the demand function (avoiding circularity), and the asymmetry between zero-shot LLM reasoning and a trained model are all addressed explicitly.
- **Self-direction and originality.** The project is planned and executed independently, organised around the project plan and risk analysis below.
- **Critical self-evaluation.** Results are evaluated against a theoretical optimum, limitations are discussed honestly, and a reflective log is kept throughout.

UI/UX mockup

The deliverable is a research tool. Its interface has two parts: a command-line entry point for launching experiments (selecting agents, seeds and scenario settings), and a results dashboard that answers the question of what the user sees. Figure 2 is a wireframe of that dashboard. It presents the headline comparison as a bar chart of total profit per agent, a line chart of price over a simulated season that exposes the strategy each agent develops, and a table of the full metrics. Much of this is produced automatically through Weights and Biases, with final figures exported using Matplotlib.

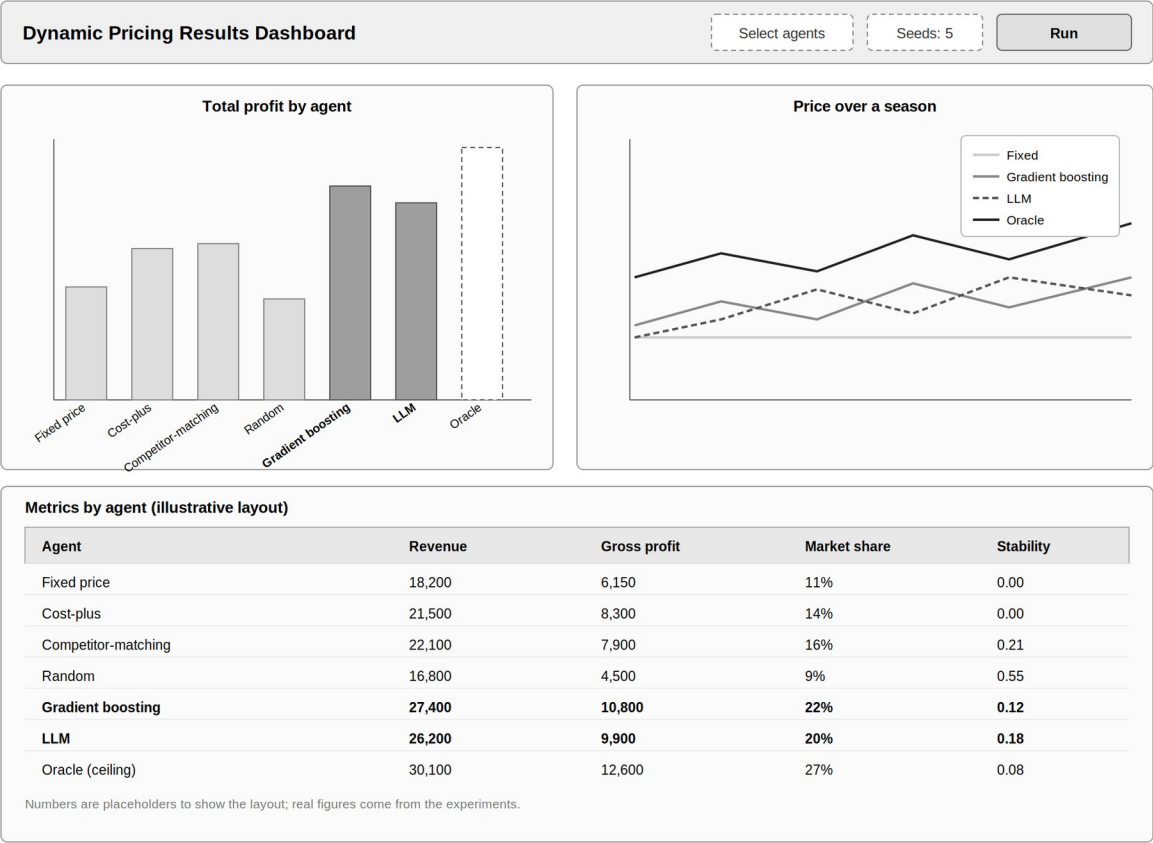


Figure 2: Wireframe of the results dashboard. The bar chart compares total profit across all agents (with the gradient-boosting and LLM agents highlighted and the oracle shown as the dashed ceiling); the line chart shows each agent's pricing over a season; and the table lists revenue, gross profit, market share and pricing stability. The numbers shown are placeholders to illustrate the layout.

Project plan

The plan spans both semesters (approximately 600 hours). The simulation environment is the critical-path dependency; the baselines precede the learning agents; and writing, the reflective log, the dissertation and the presentation are all scheduled explicitly rather than left to the end. Figure 3 shows the plan as a Gantt chart across the two semesters, with overlaps and dependencies visible.

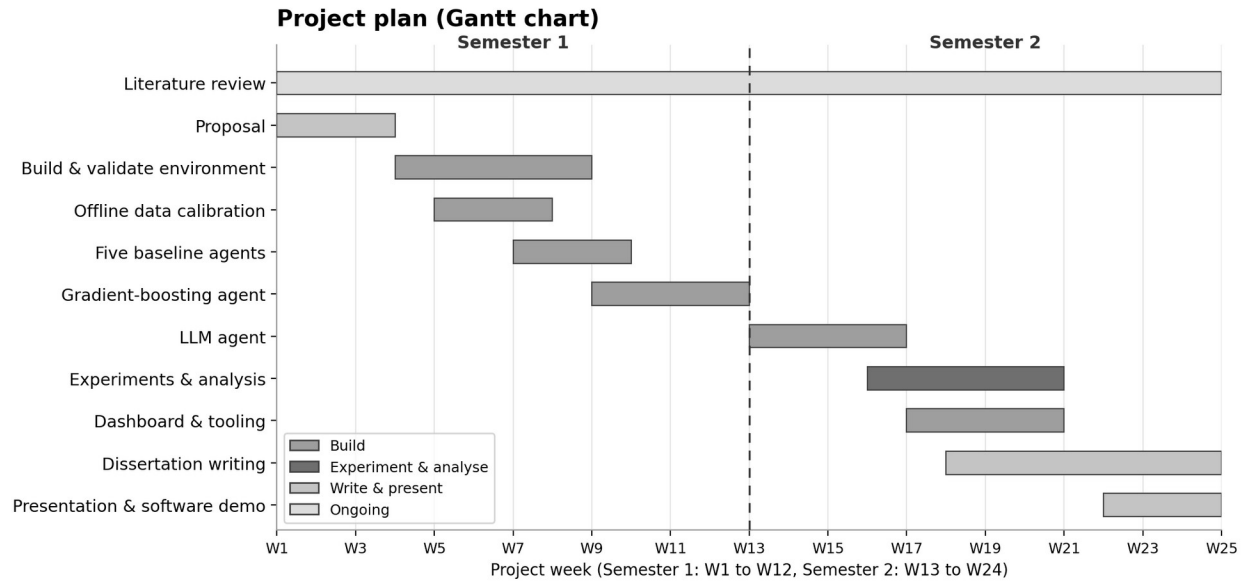


Figure 3: Project Gantt chart over 24 project weeks (Semester 1 is weeks 1 to 12, Semester 2 is weeks 13 to 24). The build phase proceeds in dependency order from the environment through to the LLM agent; experiments, dashboard work, dissertation writing and the presentation overlap through the second half; and the literature review runs throughout.

Risks and contingency plans

Risk	Contingency	Likelihood	Impact
Simulation is unrealistic, weakening validity	Calibrate from real data, sanity-check demand, run sensitivity analysis, and report the simulation-to-reality gap honestly.	Medium	High
Circularity: the demand model both defines the environment and is given to an agent	Keep the true demand a transparent function that agents must learn to estimate, never shared with an agent.	Low	High
LLM agent unreliable, costly or slow	Force structured JSON output with a parse fallback, use a smaller model, run fewer steps, cache repeated states.	Medium	Medium
Data cleaning consumes excessive time	Extract only the calibration parameters needed and timebox the work; UCI alone suffices if others slip.	Medium	Medium
Running out of time or scope creep	Build in dependency order; the LLM agent and dashboard polish are de-scopeable; track against the plan.	Medium	High

Hardware failure or data loss	Git and GitHub with frequent commits, cloud backups, and Google Colab as compute overflow.	Low	High
Programming or library problems	Use well-tested libraries, isolate new components, and test the pipeline early.	Medium	Medium

References

Chen, D. (2012) Online Retail II [Dataset]. UCI Machine Learning Repository. Available at: <https://doi.org/10.24432/C5CG6D> (Creative Commons BY 4.0).

Chen, T. and Guestrin, C. (2016) 'XGBoost: A Scalable Tree Boosting System', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785 to 794.

Den Boer, A.V. (2015) 'Dynamic pricing and learning: historical origins, current research, and new directions', Surveys in Operations Research and Management Science, 20(1 to 2), pp. 1 to 18.

GOV.UK (n.d.) UK Bank Holidays. Available at: <https://www.gov.uk/bank-holidays> (Open Government Licence).

Kastius, A. and Schlosser, R. (2021) 'Dynamic pricing under competition using reinforcement learning', Journal of Revenue and Pricing Management, 21(4), pp. 391 to 409.

Liu, J. et al. (2019) 'Dynamic Pricing on E-commerce Platform with Deep Reinforcement Learning: A Field Experiment', arXiv:1912.02572.

Office for National Statistics (2026) Retail Sales Index. Available at: <https://www.ons.gov.uk/datasets/retail-sales-index> (Open Government Licence).

Sutton, R.S. and Barto, A.G. (2018) Reinforcement Learning: An Introduction. 2nd edn. Cambridge, MA: MIT Press.

Yao, S. et al. (2023) 'ReAct: Synergizing Reasoning and Acting in Language Models', International Conference on Learning Representations (ICLR) 2023.

[Add the remaining references from your own reading, keeping this single alphabetical Harvard list, and complete the Kaggle Amazon dataset citation and its licence.]